Real-Time IoT Patient Monitoring Pipeline for Remote Health Analytics

Project Proposal (Data Engineering)

Group 2 Members:
Aiden Le Dasha Lewis Pritesh Gurjar

Course: *[CSDS 417]* **Date:** *[01/27/2026]*

1. Title and Project Team

Anticipated title: Real-Time IoT Patient Monitoring Pipeline for Remote Health Analytics.

Group 2 Member contributions (functional + technical):

- **Aiden Le (Ingestion & Data Contracts):** Define event schema/data contract; build IoT simulator and/or API collectors; implement streaming ingestion (topics/queues), schema validation, and dead-letter handling.
- **Dasha Lewis (Stream Processing & Alerts):** Implement real-time processing (windowing, dedup, late-event handling), anomaly/risk rules, and alert routing (email/SMS/webhook).
- **Pritesh Gurjar (Lake/Warehouse & Modeling):** Design Bronze/Silver/Gold layout, partitioning, and dimensional/star schema; implement batch ETL to warehouse; optimize query performance.
- **Joint Responsibility:** Analytics dashboards, CI/CD pipeline, and final documentation.

2. Objectives

2.1. Background, problem statement, and relevance

Remote patient monitoring (RPM) using IoT health devices (e.g., wearables, pulse oximeters, BP cuffs) generates continuous high-frequency data streams. In practice, these streams are noisy, heterogeneous (different vendors, sampling rates, and units), and difficult to operationalize into reliable near real-time insights. This leads to underutilization of sensor data, delayed interventions, and increased operational burden for caregivers.

Problem statement: Build an end-to-end data engineering system that ingests heterogeneous IoT health device events, standardizes them into a consistent analytical model, and produces real-time and historical views of patient health status with auditable data quality.

2.2. Research questions

1. **RQ1 (Standardization):** How can heterogeneous device signals be transformed into a consistent, analytics-ready representation (units, timestamps, patient/device IDs) while preserving provenance?
2. **RQ2 (Runtime analytics):** What streaming features (rolling averages, trend slopes, stability measures) are effective for detecting clinically relevant anomalies with low latency?
3. **RQ3 (Data quality & reliability):** Which automated checks (schema, range, completeness, duplication, timeliness) most improve trust in the pipeline outputs?

4. **RQ4 (Scalability):** How does performance (throughput/latency/cost) change with the number of simulated patients and event rates?

2.3. Context and current state of research (literature survey plan)

Existing RPM architectures emphasize streaming ingestion, event-time processing, and the need for secure/traceable health data pipelines. Standards such as HL7 FHIR provide a common way to represent clinical observations and device measurements, but practical gaps remain in scalable ingestion, data quality enforcement, and producing both real-time and BI-ready historical views in one system. Our project builds on these ideas by delivering a reproducible pipeline blueprint (data contracts, layered storage, warehouse modeling, quality gates, and runtime alerting) and quantifying pipeline performance under realistic load.

What is missing / gap: Many public examples demonstrate either (i) dashboards without robust ETL quality controls, or (ii) batch warehouses without low-latency streaming and alerting. We propose an integrated approach with explicit quality metrics, lineage, and both streaming and batch outputs.

New insights / contribution: A complete, benchmarked, and well-documented reference implementation for IoT patient monitoring that demonstrates: (1) heterogeneous ingestion + standardization, (2) near real-time anomaly detection, (3) lake + warehouse outputs for analytics, and (4) measurable data quality and performance.

3. Materials and Methods

3.1. Proposed Architecture

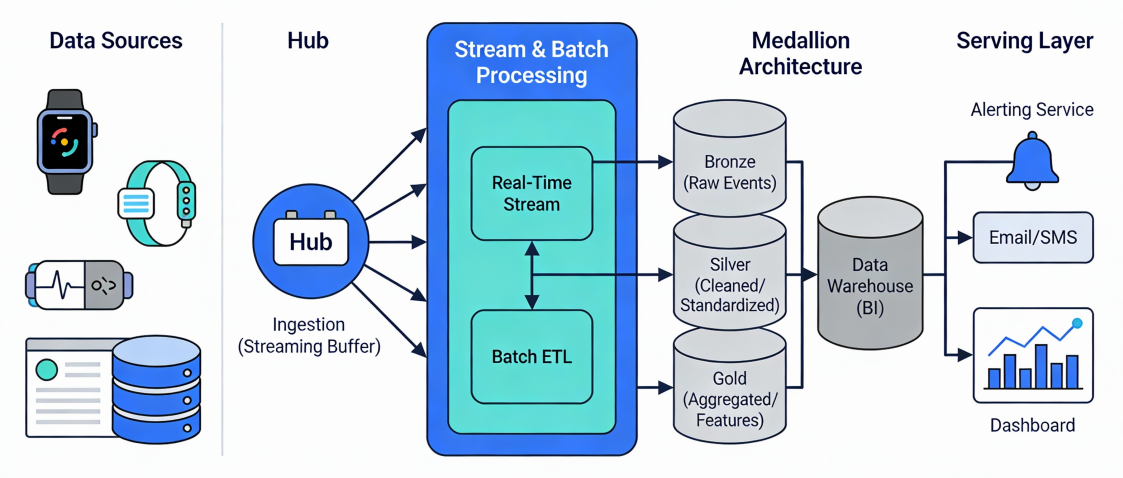

Figure 1: High-level architectural data flow for the Real-Time Patient Monitoring Pipeline.

3.2. Data sources

We will use multiple heterogeneous sources to satisfy the course requirement for varied formats:

- **Streaming JSON events:** Synthetic IoT device telemetry per patient (HR, SpO_2, temperature, BP, activity).

- **Reference tables (CSV/SQL):** Patient profile (age band, baseline ranges), device metadata (vendor, sensor type, sampling rate), and alert thresholds.
- **Optional API source:** Public weather/air-quality API (for contextual enrichment) or public health calendar/events feed (for drift/seasonality experiments).

3.3. Data ingestion

- Implement a device simulator that publishes patient readings at configurable rates (e.g., 1–10 Hz per patient) with injected anomalies (hypoxia episodes, tachycardia, fever).
- Ingest events through a streaming layer (message queue / log-based broker) with schema checks and a dead-letter path for malformed records.
- Enforce event-time semantics: handle late/out-of-order events and deduplicate using (patient_id, device_id, timestamp, sequence_id).

3.4. Storage design

We will implement a layered architecture:

- **Bronze (raw):** Append-only raw events (JSON/Parquet) partitioned by date/hour and optionally by device/vendor.
- **Silver (clean):** Validated, standardized units, normalized timestamps, resolved patient/device keys, quality flags.
- **Gold (curated):** Analytics-ready aggregates and feature tables, e.g., `latest_vitals`, `patient_minute_feature` `alert_events`, `daily_summary`.

3.5. Processing pipelines and distributed environment

- **Streaming processing:** Windowed feature computation (e.g., 1/5/15-minute windows), trend detection, and rule-based (or lightweight ML) anomaly scoring.
- **Batch ETL:** Nightly compaction, backfill, and warehouse loads; compute historical baselines and patient-specific thresholds.
- **Distributed processing:** Run transformations on a distributed engine (cloud cluster or managed service) to demonstrate scalability.
- **Serving for analytics/ML:** Publish curated tables to a data warehouse for BI; expose an API for near real-time "patient status" reads.

3.6. Visualization

- Real-time dashboard: current vitals, alert banner, and rolling trend charts per patient.
- Operational dashboard: ingestion rate, end-to-end latency, late-event rate, and data quality KPIs.
- Analytical dashboard: cohort summaries (e.g., alerts per day, distribution of vitals, adherence/uptime per device).

3.7. Evaluation metrics

- **Latency:** time from event ingestion to alert emission and to dashboard update.
- **Throughput:** events/sec sustained under load.
- **Data quality KPIs:** validity rate, completeness, duplication, timeliness, and schema conformance.
- **Reliability:** pipeline success rate, recoverability (replay from broker), idempotent writes.

3.8. Achievability and timeline (milestones)

- **Week 1:** Finalize data contract + simulator; baseline ingestion to Bronze.
- **Week 2:** Implement Silver transformations + quality checks; basic dashboards.
- **Week 3:** Streaming features + alerting pipeline; alert audit table.
- **Week 4:** Gold tables + warehouse modeling (star schema) + BI queries.
- **Week 5:** Scaling tests, performance tuning, reliability (replay, idempotency), monitoring.
- **Week 6:** Final integration, documentation, demo script, and final report draft.

4. Results (Expected Outcomes)

We expect to deliver a working end-to-end pipeline that:

- Continuously ingests IoT vitals streams and produces cleaned, standardized Silver data plus curated Gold aggregates.
- Detects anomaly events in near real-time and logs alerts with provenance (which rule/feature triggered).
- Supports analyst-style queries in a warehouse (trend analysis, cohort summaries) and provides a live dashboard.
- Reports quantified performance (latency/throughput) and data quality metrics across multiple simulated load levels.

5. Discussion

This project contributes a practical reference architecture for RPM-style analytics that integrates streaming and batch pipelines while emphasizing data quality, reproducibility, and measurable system performance. It can serve as a basis for future extensions such as FHIR-native storage, advanced ML risk modeling, personalization of thresholds, or multimodal fusion (sensor + patient text). It also enables discussion of key data engineering tradeoffs: event-time handling, schema evolution, partitioning strategies, and balancing alert sensitivity vs false positives.

6. Conclusion

We propose an achievable, course-timeline-feasible end-to-end data engineering project that integrates heterogeneous IoT patient data sources, implements robust ingestion and layered storage,

and produces both real-time monitoring and historical analytics outputs. Limitations include use of synthetic data (no clinical validation) and simplified alert logic; however, the pipeline design, operational metrics, and reproducibility will demonstrate strong data engineering competence and provide a credible foundation for future research and development.